

# ClaimFlow – Simple Build Guide (Hackathon MVP)

This document is a **practical, step-by-step guide** to help you build the **Insurance Claim Companion Agent** within a hackathon timeframe (6–8 hours). It focuses on **what to build first**, **how components connect**, and **what to demo**.

---

## 1. Product Goal (Keep This Fixed)

**Goal:** Build a web app that **reads an insurance policy and actively manages the claim process** for a user.

**In one line:**

Upload policy → describe incident → agent creates and manages claim steps.

---

## 2. End User Experience (What the user sees)

### User Journey

1. Upload insurance policy (PDF)
2. Enter claim details (what happened)
3. Click **“Start Claim”**
4. See a **claim checklist** created automatically
5. Upload documents step-by-step
6. Agent updates status, flags issues, and drafts follow-ups

### UI Components (Minimal)

- Policy upload section
  - Incident description form
  - Claim checklist (To Do / Done / Blocked)
  - Document upload buttons
  - Status messages (Valid / Missing / At Risk)
- 

## 3. Core Agent Responsibilities (Very Important)

The agent is responsible for **managing the process**, not just answering questions.

### What the Agent Must Do

- Read policy rules using RAG
- Decide what steps are required for the claim

- Create a task workflow
  - Validate uploaded documents
  - Track deadlines
  - Trigger reminders or escalation
- 

## 4. Technical Architecture (Simple)

### Frontend

- Streamlit (recommended for speed)
- Shows checklist, uploads, and status

### Backend

- Python + FastAPI
- Handles agent logic and state

### AI / Agent Layer

- Gemini API (LLM)
- LangGraph (for agent flow)

### Storage

- Vector DB: FAISS or Chroma (policy documents)
  - SQLite or in-memory dict (claim state)
- 

## 5. Tools Used by the Agent

These tools make the system **agentic**.

### Tool 1: Policy Reader (RAG)

**Input:** Policy PDF **Output:** Extracted rules (coverage, docs, timelines)

---

### Tool 2: Claim Workflow Generator

**Input:** Policy rules + incident description **Output:** Structured task list

Example: - Notify insurer - Upload hospital bill - Upload discharge summary

---

### Tool 3: Document Validator

**Input:** Uploaded document + expected doc type **Output:** Accepted / Rejected / Missing

---

### Tool 4: Deadline Tracker

**Input:** Task + deadline **Action:** Flag overdue tasks and trigger reminder

---

### Tool 5: Email / Appeal Generator

**Input:** Claim status + issue **Output:** Drafted follow-up or appeal email

---

## 6. Agent Flow (Logical Steps)

1. User uploads policy
  2. Agent reads policy using RAG
  3. User describes incident
  4. Agent decides if claim is valid
  5. Agent creates claim workflow
  6. User uploads documents
  7. Agent validates and updates task status
  8. Agent monitors deadlines
  9. If rejected → agent creates appeal step
- 

## 7. Data Models (Keep Simple)

### Claim

- claim\_id
- policy\_id
- incident\_description
- status

### Task

- task\_id
- name
- status (Pending / Done / Blocked)
- deadline

## Document

- document\_type
  - uploaded (Yes/No)
  - validation\_status
- 

## 8. What to Build First (Priority Order)

### MUST HAVE (Core MVP)

1. Policy upload + RAG
2. Claim checklist auto-generation
3. Task status tracking
4. Document upload & validation

### NICE TO HAVE (If time allows)

- Appeal email generation
  - Deadline reminders
  - Risk flagging
- 

## 9. Demo Script (2 Minutes)

1. Upload policy PDF
  2. Enter incident details
  3. Show agent-created checklist
  4. Upload one document
  5. Show task auto-marked as done
  6. Show missing document warning
  7. Generate appeal email
- 

## 10. How to Explain This to Judges (Simple)

"Our agent reads insurance policies and actively manages the claim process. It creates tasks, checks documents, tracks deadlines, and prepares appeals—so users don't lose money due to process mistakes."

---

## 11. What This Is NOT (Avoid Overclaiming)

- Not legal advice
- Not auto-submitting claims to insurers

- Not guaranteeing claim approval

It is **decision-support + process automation**.

---

## 12. Success Criteria

You have succeeded if: - A checklist appears automatically - Task status changes without user asking - Agent flags missing documents - Agent produces an actionable output (email / escalation)

---

**This is enough to win attention in a hackathon.**